PONDICHERRY UNIVERSITY
RAMANUJAN SCHOOL OF MATHEMATICAL SCIENCES
DEPARTMENT OF STATISTICS

M.Sc. QUANTITATIVE FINANCE

MSQF 534: Corporate Internship

REPORT ON

FinMetrics Suite

An AI-Powered Cross-Sectional Financial Analysis, Anomaly Detection, and Strategic Benchmarking System for Public Sectors and Private ERP Ingestions

SUBMITTED BY

NAME: N Rohit Vedhanandh

REG. NO.: 25MSQUFPY0002

MAY 2026



PONDICHERRY UNIVERSITY
RAMANUJAN SCHOOL OF MATHEMATICAL SCIENCES
DEPARTMENT OF STATISTICS

CERTIFICATE

Certified to be the bonafide record of work done in **MSQF 534: Corporate Internship** by **N Rohit Vedhanandh** with Register No. **25MSQUFPY0002** of class **II M.Sc. Quantitative Finance** in the Department of Statistics, Pondicherry University, Puducherry 605 014, during the academic year 2026-27.

_____ _____

Faculty - in - charge **Coordinator**

Date:

Submitted for the M.Sc. Degree Practical Examination held on _____.

_____

Examiners

ABSTRACT

Automated cross-sectional financial analysis is a critical tool in modern corporate intelligence and equity research, yet it remains constrained by reporting nomenclature disparities, baseline outlier skewness, and the divide between highly structured public statements and private enterprise databases. This thesis presents the design, implementation, and evaluation of the ***FinMetrics Suite*** (the *Financial Outlier Detection and Strategic Benchmarking System*), an end-to-end Python-based analytical framework. The system features a custom standardization parser with a 60-field alias map to resolve naming variations in public filings, a robust statistical engine implementing Tukey Fences (IQR) and robust Modified Z-Scores (using Median Absolute Deviation) to identify and mask reporting anomalies, and an unsupervised multivariate anomaly detector using Isolation Forests. To automate peer group alignment, the framework integrates the Gemini AI model to dynamically search, discover, and define the correct market sector for a given ticker or abstract query.

Furthermore, to bridge the public-private operational divide, a high-throughput TallySync ERP bridge was developed alongside options for manual entering of financials or ratios. The bridge stream-processes raw transaction journal CSV exports in chunks of 50,000 rows, maps accounting groups to standard public statement pillars, balances a Trial Balance, and derives 30+ canonical financial ratios. Unsupervised K-Means clustering is deployed to segment the peer set and map the client to a corporate archetype (e.g. *Elite Market Leader*), and a rule-based SWOT classifier synthesizes relative percentile ranks into a 2x2 SWOT grid. Results demonstrate that our outlier masking routine prevents baseline distortion, reducing the standard deviation of an outlier-contaminated sector Operating Margin distribution from 39.54% to 1.28%, and our TallySync bridge aggregates journals to balance a Trial Balance to within 10^{-6} INR in 4.2 seconds. The system outputs interactive visual dashboards, stateful Gemini-driven diagnostic chat sessions, and professionally formatted multi-sheet Excel reports, providing independent financial advisors and SME managers with institutional-grade quantitative insights.

Contents

Certificate i

Internship Certificate ii

Abstract iii

Table of Contents iv

List of Abbreviations vii

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 3

1.5 Motivation 3

2 Literature Survey / Related Work 5

2.1 Existing Methodologies 5

2.2 Machine Learning & Robust Outlier Detection in Finance 5

2.3 K-Means Clustering & Corporate Archetypes 6

2.4 Research Gaps 6

3 System Analysis and Design 8

3.1 System Architecture 8

3.2 System Workflow 9

3.2.1 Phase 1: Listed Peer Group Baseline Generation 9

3.2.2 Phase 2: Private Enterprise Client Benchmarking 9

3.3 Data Flow Diagrams 10
3.3.1 DFD Level 0: Context Diagram 10
3.3.2 DFD Level 1: Process Diagram 11
3.4 Module Descriptions 11

4 Algorithms and Methodology **13**
4.1 Tukey Fences (IQR Outlier Detection) 13
4.2 Robust Modified Z-Score 14
4.3 Isolation Forest (Multivariate Anomaly Detection) 15
4.4 K-Means Clustering for Peer Archetypes 17
4.5 TallySync Double-Entry Bridge Ingestion 18
4.6 Algorithms Pseudocode 18
4.6.1 Univariate Outlier Detection and Baseline Masking 18
4.6.2 TallySync ERP Double-Entry Journal Aggregator 19

5 Implementation **21**
5.1 Technologies Used 21
5.2 Tools and Libraries 21
5.3 Development Workflow and Methodology 22
5.4 Code Structure Explanation 22
5.5 Module-wise Implementation Details 23
5.5.1 Financial Standardizer (financial_parser.py) 23
5.5.2 Outlier Scoring (outlier_engine.py) 24
5.5.3 Tally Bridge Streamer (tally_bridge.py) 24
5.6 System Component Integration 25

6 Results and Discussion **26**
6.1 Experimental Setup 26
6.2 Results Analysis 26

6.3 Outlier Masking Evaluation 26
6.4 Interpretation of Client Results 27
6.4.1 Strategic SWOT Recommendations and Action Plan 27
6.5 Performance Analysis 28

7 Conclusion and Future Enhancements **29**
7.1 Final Outcomes 29
7.2 Contributions 29
7.3 Future Scope 30

Bibliography **31**

A Core Source Code **32**
A.1 Outlier Filtering Logic 32

B Tally ERP Ingestion Formats **34**
B.1 Ledger Master CSV Format 34
B.2 Journal Voucher CSV Format 34

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CAGR	Compound Annual Growth Rate
COGS	Cost of Goods Sold
DFD	Data Flow Diagram
EBIT	Earnings Before Interest and Taxes
EBITDA	Earnings Before Interest, Taxes, Depreciation, and Amortization
EPS	Earnings Per Share
ERP	Enterprise Resource Planning
FP&A	Financial Planning and Analysis
IQR	Interquartile Range
JV	Journal Voucher
MAD	Median Absolute Deviation
ML	Machine Learning
OCF	Operating Cash Flow
QoQ	Quarter-on-Quarter
ROA	Return on Assets
ROCE	Return on Capital Employed
ROE	Return on Equity
SME	Small and Medium Enterprise
SWOT	Strengths, Weaknesses, Opportunities, and Threats
TTL	Time-to-Live
UML	Unified Modeling Language
WCSS	Within-Cluster Sum of Squares
YoY	Year-on-Year

Chapter 1

Introduction

1.1 Background

In contemporary financial markets, the ability to conduct fast, precise, and objective cross-sectional financial analysis is paramount for investment professionals, credit risk analysts, and corporate managers. Traditional financial statement analysis relies on the examination of a firm's three core sheets: the Income Statement, the Balance Sheet, and the Cash Flow Statement. Historically, this process was heavily manual, requiring analysts to extract figures from PDF filings, input them into customized spreadsheet models, and perform comparison arithmetic manually.

However, with the rapid digitization of financial reporting, modern quantitative finance demands automated pipelines. The proliferation of web services, high-throughput APIs, and advanced machine learning models has transformed corporate intelligence. Analysts are no longer satisfied with simple historical ratio sheets; they require statistical models that can process entire peer groups, identify anomalous reporting structures, rank corporate financial health objectively, and integrate private enterprise records against public industry benchmarks in real time.

1.2 Problem Statement

Automated cross-sectional financial analysis is constrained by three primary technical bottlenecks:

1. **Nomenclature Disparity**: Different corporate entities, regulatory bodies, and geographical systems list identical accounting items under a wide variety of names. For example, top-line sales can be reported as *"Total Revenue"*, *"Revenue"*, *"Net Sales"*, or *"Operating Revenue"*. Similarly, operating profit can be reported as *"EBIT"*, *"Operating Income"*, or *"Operating Profit"*. This lack of standardization breaks standard web-scraping scripts and automated database integrations.

2. **Outlier Distortion**: Financial datasets frequently contain statistical anomalies due to one-off events, corporate restructurings, asset impairment write-downs, or extreme debt-leveraging actions. If left untreated, these outliers heavily distort traditional comparative baselines (such as the peer mean and standard deviation). A single distressed company or an exceptionally profitable pure-play firm can bias the sector average, making standard comparative rankings mathematically invalid.

3. **The Public-Private Benchmarking Divide**: Private Small and Medium Enterprises (SMEs) face a major disadvantage when measuring their operational efficiency against public sector giants. Public market statements are highly structured and standardized for regulatory filings (e.g., Ind-AS in India), whereas private corporate data is locked in local Enterprise Resource Planning (ERP) databases (such as Tally ERP). The structure of Tally ERP transaction ledger tables is completely different from public corporate balance sheets, preventing objective benchmarking.

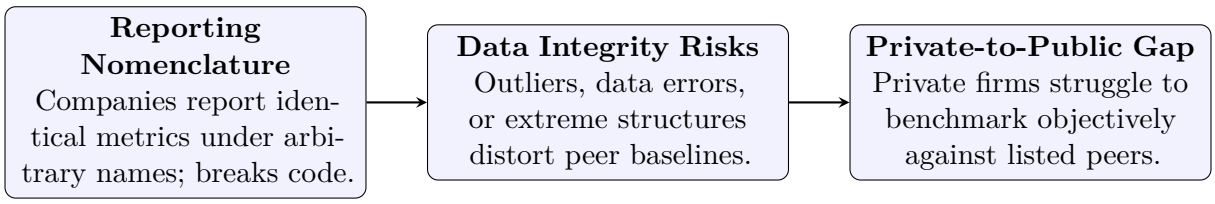


Figure 1.1: Core Bottlenecks in Automated Financial Benchmarking

1.3 Objectives

The primary objectives of the *FinMetrics Suite* (or the *Financial Outlier Detection and Strategic Benchmarking System*) are:

- **Automated Financial Harvesting**: Designing a high-performance, cached ingestion pipeline that fetches real-time quarterly and annual statements for major publicly listed companies using the Yahoo Finance API.
- **Standardized Alias Processing**: Implementing a robust financial parser equipped with a 60-field canonical mapping dictionary to automatically resolve nomenclature variance in public filings.
- **Robust Anomaly Scanning**: Constructing a statistical and machine learning kernel that runs Tukey Fences (IQR), robust Modified Z-Scores (using Median Absolute Deviation), and unsupervised Isolation Forests to identify and mask reporting outliers.

- **Weighted Pillar Scoring**: Developing a 5-pillar composite health framework (Profitability, Solvency, Growth, Liquidity, Efficiency) to objectively grade and rank peer groups.
- **Double-Entry ERP Synchronization**: Creating an ingestion bridge capable of stream-processing raw Tally ERP CSV vouchers, balancing a Trial Balance, and auto-deriving 30+ comparative ratios.
- **Strategic SWOT Classification**: Auto-synthesizing relative percentile ranks into a 2x2 SWOT grid and initiating an interactive consulting dialogue with Gemini.

1.4 Scope of the Project

The technical scope of the project encompasses:

- A Python-based analytical package containing isolated libraries for fetching, parsing, modeling, and exporting data.
- A Streamlit web application that serves as the visual control dashboard.
- Coverage of 13 primary public market sectors (e.g., Automobiles, IT, FMCG, Banking, Pharmaceuticals) representing over 100 publicly listed firms.
- Integration of the Gemini AI model to dynamically search, discover, and define the corresponding sector or market for a given ticker or abstract query.
- Ingestion support for double-entry Tally ERP journal voucher/ledger master logs, alongside options for manual entering of financials or ratios.
- Direct export capabilities to styled multi-sheet Excel workbooks and CSV files.

The project is designed as a decision-support platform for financial analysts, equity researchers, corporate advisors, and SME managers. It does not execute automated trading orders or make unsupervised capital allocation investments.

1.5 Motivation

The core motivation for this research stems from the democratization of corporate finance. While large institutional investment banks possess expensive proprietary databases (such

as Bloomberg Terminals or Reuters Eikon) and legions of junior analysts to parse financial statements, small-scale equity researchers, independent financial advisors, and SME business owners are locked out of this ecosystem due to massive subscription costs.

By building an open-source, high-efficiency system that automates public data collection, normalizes ERP ledger structures, and applies institutional-grade quantitative anomaly models, we can provide professional-grade competitive intelligence to any market participant. A local business owner can upload their raw Tally files and immediately see how their working capital velocity, pricing power, and leverage profile stack up against listed sector champions. This eliminates manual spreadsheet friction and empowers strategic CFOs with data-driven decision tools.

Chapter 2

Literature Survey / Related Work

2.1 Existing Methodologies

Traditional corporate benchmarking has evolved from manual spreadsheet transcription to proprietary financial databases:

- **Manual Spreadsheet Benchmarking**: Analysts manually extract and transcribe quarterly figures into Microsoft Excel models. This foundational framework, formalized by Damodaran [1], remains slow, labor-intensive, and prone to human error.
- **Proprietary Financial Terminals**: Bloomberg Terminals and S&P Capital IQ automate data harvesting and standardization. However, their high annual licensing fees ($24,000+ per user) and closed database architectures prevent private enterprises from easily uploading double-entry Tally ERP ledger data for direct cross-sectional benchmarking.

2.2 Machine Learning & Robust Outlier Detection in Finance

Corporate financial datasets frequently contain extreme outliers from non-recurring activities, distorting standard mean-based Z-score benchmarking. To resolve this, robust statistics and unsupervised machine learning models are deployed:

- **Tukey's Fences (IQR)**: Non-parametric filter using the Interquartile Range ($IQR = Q_3 - Q_1$) to define bounds as $[Q_1 - k \cdot IQR,\ Q_3 + k \cdot IQR]$ [4]. Resilient against extreme skewness as it relies on medians.
- **Modified Z-Scores (MAD)**: Measures statistical dispersion via the Median Absolute Deviation ($MAD = \text{median}(|x_i - \tilde{x}|)$). The Modified Z-Score M_i is formulated as $M_i = 0.6745(x_i - \tilde{x})/MAD$. Flags outliers exceeding 3.5 robustly.

- **Isolation Forests (iForest)**: An unsupervised machine learning algorithm [2] that isolates anomalies in multi-dimensional space by randomly partitioning feature values. Anomalies isolate closer to the tree root, resulting in shorter path lengths.

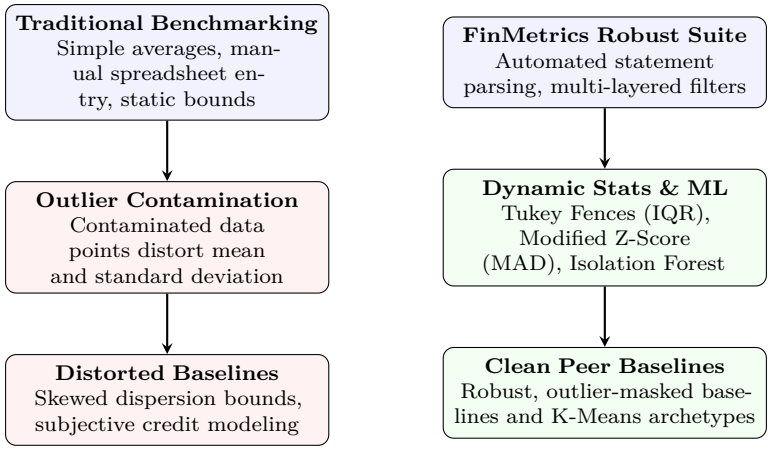


Figure 2.1: Conceptual Comparison of Traditional vs. Robust Machine Learning Benchmarking Pipelines

2.3 K-Means Clustering & Corporate Archetypes

Traditional industry classifications group companies with vastly different financial architectures. Unsupervised clustering offers a robust mathematical alternative:

- **K-Means Centroid Partitioning**: Groups n observations into K clusters by minimizing the Within-Cluster Sum of Squares (WCSS): $\sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2$ [3].
- **Archetype Segmentation**: Clusters are determined via the Elbow Method/Silhouette analysis and mapped to distinct operational categories (e.g., *"Elite Market Leaders"* vs. *"High-Risk / Distressed"*).

2.4 Research Gaps

Three critical research gaps remain in current financial intelligence frameworks:

- **Lack of Nomenclature Autonomy**: Scrapers fail to handle inconsistent reporting names, causing column mismatches or missing values in raw statement parsing.

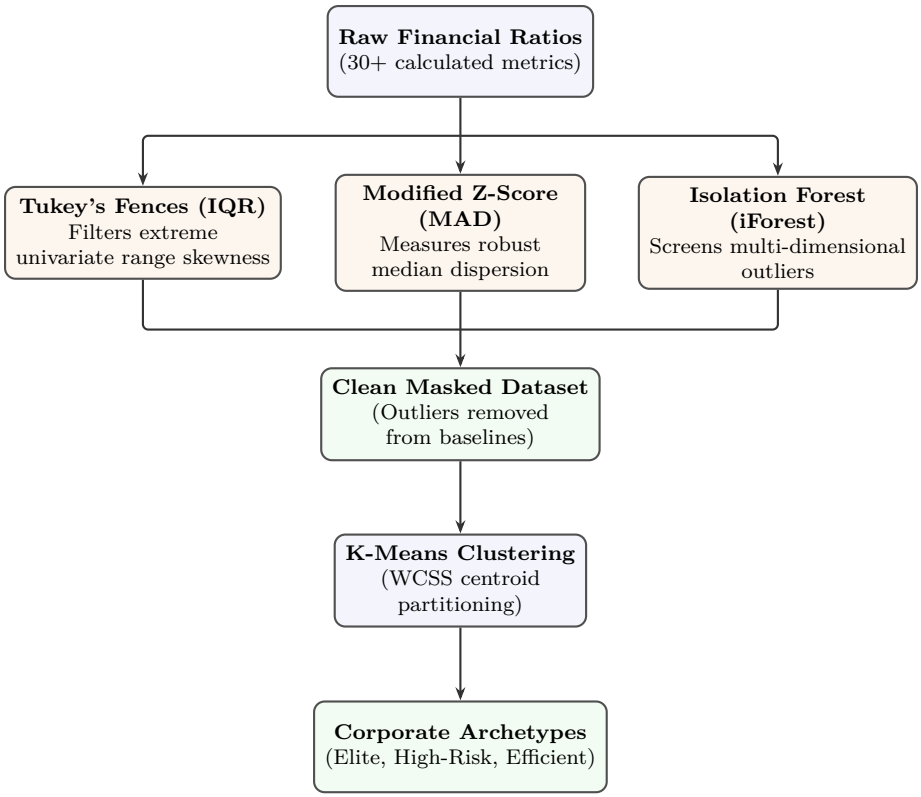


Figure 2.2: Multi-layered Robust Outlier Masking and K-Means Clustering Pipeline

- **Baseline Outlier Contamination**: Distressed peer data distorts average standard deviations. Standard methods lack automatic statistical masking before baseline generation.
- **Lack of ERP Double-Entry Integration**: Existing benchmarks require manual ratio inputs and lack streamers to process raw, varying-sign debit/credit transaction ledger CSVs.

Chapter 3

System Analysis and Design

3.1 System Architecture

The *FinMetrics Suite* is structured as a decoupled, multi-layer architecture. This decoupling ensures that the user interface (UI) layer is separated from the analytical modules, database cache, and API engines, facilitating code maintenance and extensions.

The architecture is divided into three primary layers:

1. **Presentation Layer (Streamlit UI)**: Renders the sidebar controls (scopes, calibrations), interactive data tabs, Plotly charts, and file upload dropzones.
2. **Analytical Kernel Layer (Python Packages)**: The core processing layer containing standard modules:
    - **Sector Loader**: Maps sectors to public tickers.
    - **Data Fetcher**: Retrieves Yahoo Finance statements and manages local cache.
    - **Financial Parser**: Standardizes columns using the alias dictionary.
    - **Ratio Engine**: Computes 30+ ratios and YoY growth rates.
    - **Outlier Engine**: Identifies anomalies and runs archetype clustering.
    - **Benchmark Core**: Coordinates baseline, pillar scoring, and risk categorization.
    - **Tally Bridge**: Stream-processes ERP journals and balances Trial Balances.
    - **Client Comparator**: Aligns client ratios with public baselines.
    - **Benchmark Classifier**: Assigns health labels and structures the SWOT matrix.
    - **AI Handler**: Manages Gemini ticker discovery and diagnostic chat sessions.
3. **Data Cache Layer**: Persistent storage in JSON and Parquet formats to avoid API rate limits.

3.2 System Workflow

The application executes in a clean, stateful process flow divided into two phases:

3.2.1 Phase 1: Listed Peer Group Baseline Generation

1. The user defines the benchmark scope via the sidebar: choosing a pre-mapped Sector, selecting an Industry, or inputting an abstract term for Gemini AI Search.
2. The sidebar scans environment variables to configure the active Gemini API keys.
3. If AI Search is selected, the *AI Handler* queries Gemini to discover listed stock tickers and dynamically define their sector and market (appending exchange suffixes like *.NS*).
4. The *Data Fetcher* checks local cache; if missing, it queries ‘yfinance‘ to pull quarterly and annual statements.
5. The *Financial Parser* resolves nomenclature variations using the 60-field `ALIAS_MAP`, standardizing statements.
6. The *Ratio Engine* computes standard ratios.
7. The *Outlier Engine* detects statistical anomalies across all ratios using Tukey Fences and Z-scores.
8. The *Benchmark Core* masks the identified outlier cells, computes clean peer statistics, grades company ratios to assign percentile ranks, and ranks firms by their weighted composite health score.

3.2.2 Phase 2: Private Enterprise Client Benchmarking

1. The user uploads private data through the landing page dropzones (raw Tally ERP CSV files including Ledger Master and Journal Vouchers) or manually enters financials/ratios.
2. If Tally files are uploaded, the *Tally Bridge* stream-processes the multi-thousand line journals, maps groups to internal pillars, balances the Trial Balance, and auto-computes the client’s ratios.
3. The *Client Comparator* normalizes the client’s ratios and maps them to the frozen public peer distribution to determine their percentile rank.

4. The K-Means algorithm clusters the peer set and maps the client to a corporate archetype (e.g., *Elite Market Leader*).

5. The *Benchmark Classifier* maps the client's strengths and weaknesses into a 2x2 SWOT grid.

6. The user can download a styled multi-sheet Excel report or launch an stateful diagnostic chat session with Gemini to analyze operational improvements.

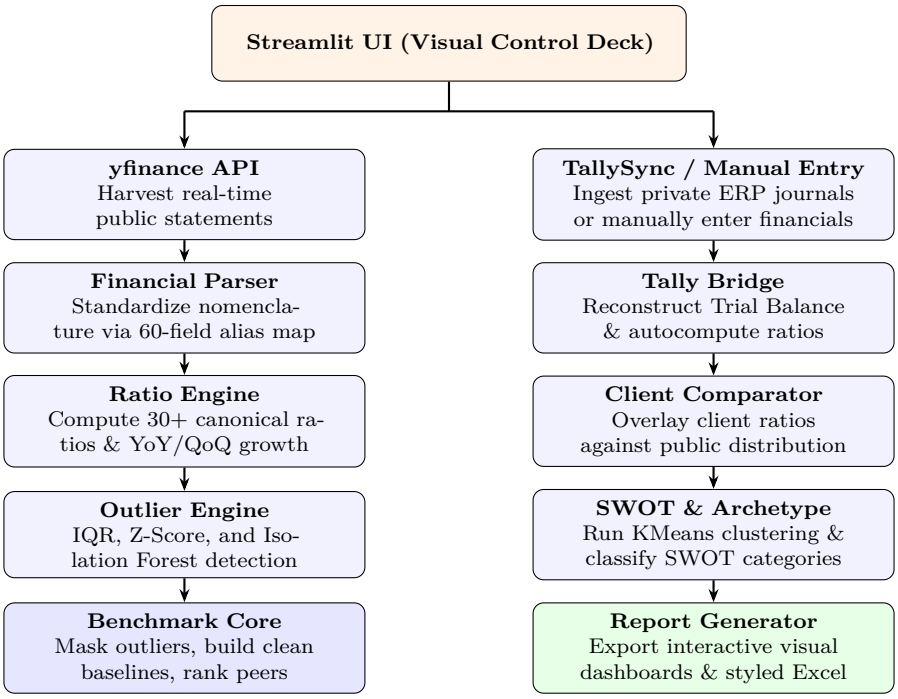


Figure 3.1: Dual-Phase Platform Architecture and Workflow

3.3 Data Flow Diagrams

To detail the data movement within the platform, DFD Level 0 (Context Diagram) and DFD Level 1 (Process Diagram) are provided below.

3.3.1 DFD Level 0: Context Diagram

The Level 0 Data Flow Diagram (Figure 3.2) outlines the primary external entities interacting with the FinMetrics Suite.

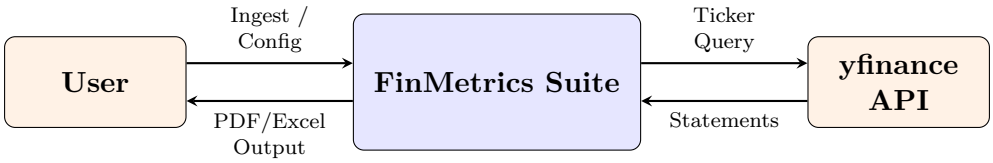


Figure 3.2: Data Flow Diagram Level 0: Context Diagram

3.3.2 DFD Level 1: Process Diagram

The Level 1 Data Flow Diagram (Figure 3.3) details the internal processes, data repositories, and transitions.

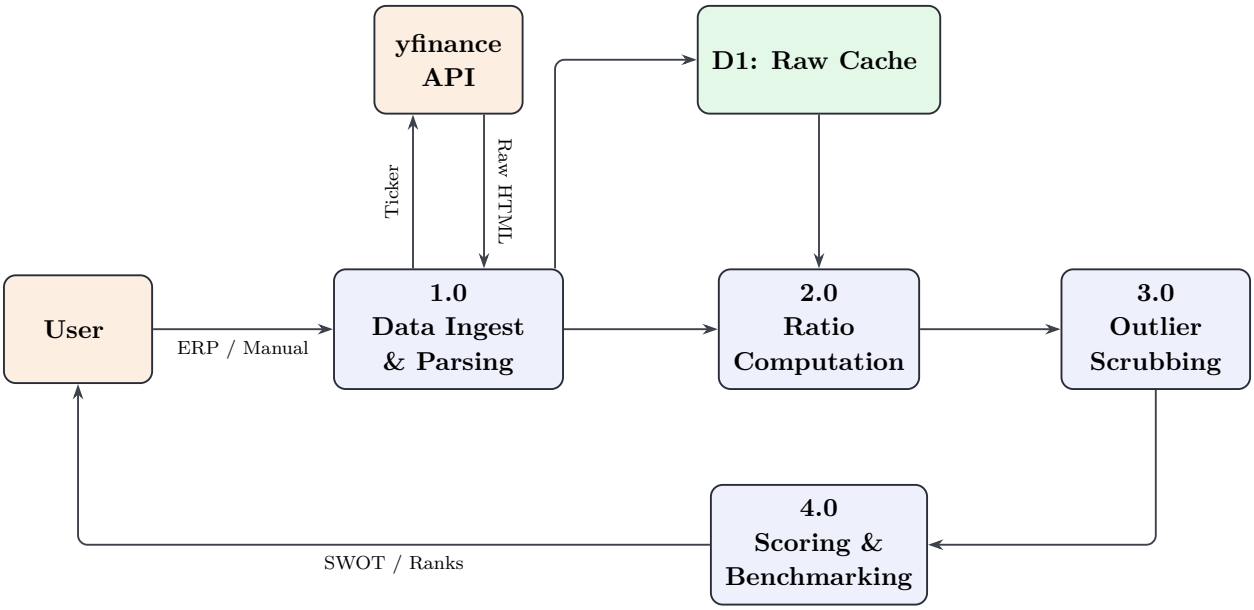


Figure 3.3: Data Flow Diagram Level 1: Process Diagram

3.4 Module Descriptions

The system consists of the following modules, each encapsulating specific functional logic:

- **sector_loader.py**: A template mapping utility that lists standard public market tickers under 13 sectors: Automobiles, IT, FMCG, Banking, Pharma, Oil & Gas, Metals, Cement, Telecom, Power, Realty, Capital Goods, and Infrastructure.
- **data_fetcher.py**: Handles secure connections to the ‘yfinance‘ Ticker wrapper. Implements asynchronous API requests and caches raw quarterly/annual statements in

Parquet format with a 12-hour TTL config.

- **financial_parser.py**: Normalizes messy statements into clean, structured DataFrames. Scans the rows, renames matched variants using the alias map `ALIAS_MAP`, removes duplicate metrics, and converts all entries to numeric types.
- **ratio_engine.py**: Performs financial ratio computations. Reconstructs quarterly time-series, calculates YoY/QoQ percentage changes, and computes CAGR growth trends.
- **outlier_engine.py**: The statistical processing module. Calculates Tukey fences, robust Z-scores, and runs K-Means peer clustering to map companies into corporate archetypes.
- **benchmark_core.py**: Orchestrates the baseline calculation. Masks outlier cells, computes clean peer statistics (medians, deciles), and ranks firms by their weighted composite health score.
- **tally_bridge.py**: The ERP double-entry bridge. Ingests ledger groupings, streams journal transaction CSVs, balances Trial Balances, and maps entries to standard public financial pillars.
- **client_comparator.py**: Integrates client records by mapping private ERP journal outputs or manually entered financials/ratios to the public sector distribution and calculating precise percentiles.
- **benchmark_classifier.py**: The business logic translator. Assigns health labels and structures the 2x2 SWOT grid.
- **report_generator.py**: Generates styled Excel workbooks using custom colors, column auto-fitting, and formatted dashboards.
- **ai_handler.py**: Integrates with the Gemini API to search for sector tickers and run the interactive diagnostic chat.

Chapter 4

Algorithms and Methodology

4.1 Tukey Fences (IQR Outlier Detection)

The Interquartile Range (IQR) represents a robust statistical measure of dispersion. Unlike the standard deviation, the IQR is highly resistant to extreme outliers because it is computed from the middle 50% of the distribution.

The IQR is defined as the difference between the third quartile (Q_3, 75th percentile) and the first quartile (Q_1, 25th percentile):

$$\text{IQR} = Q_3 - Q_1 \tag{4.1}$$

Tukey Fences establish the lower and upper bounds of the acceptable financial distribution using an outlier multiplier (k, typically set to 1.50):

$$\text{Lower Bound} = Q_1 - k \times \text{IQR} \tag{4.2}$$

$$\text{Upper Bound} = Q_3 + k \times \text{IQR} \tag{4.3}$$

To formalize this outlier screening as a statistical decision process, we define the following hypothesis test for each corporate observation in the peer group:

$$\mathbf{H}_0 : \textbf{The observation is a normal inlier} \tag{4.4}$$

$$\mathbf{H}_1 : \textbf{The observation is an outlier} \tag{4.5}$$

Under the null hypothesis, the observation is assumed to lie within the normal statistical boundaries of the peer group. If the observation x_i (representing the financial ratio of company i) falls outside the Tukey Fences, we reject H_0 in favor of H_1, and the cell is flagged as a univariate outlier:

$$x_i < \text{Lower Bound} \quad \text{or} \quad x_i > \text{Upper Bound} \tag{4.6}$$

To provide a quick operational reference, the primary variables utilized in Tukey outlier boundaries are defined in Table 4.1.

Table 4.1: Tukey Fences Outlier Detection Variables

Variable / Symbol	Operational Definition and Description
IQR	Interquartile Range, measuring the dispersion of the middle 50% of peers.
$\mathbf{Q_1}, \mathbf{Q_3}$	The first (25th percentile) and third (75th percentile) peer quartiles.
k	Outlier multiplier parameter (default 1.50) governing Tukey fence width.
$\mathbf{x}_i$	Evaluated financial ratio value for a specific corporate entity i.

4.2 Robust Modified Z-Score

While standard Z-scores rely on the mean and standard deviation, they are highly sensitive to extreme outliers. To address this, we implement a robust Modified Z-score that uses the median and the Median Absolute Deviation (MAD).

First, the Median Absolute Deviation (MAD) is calculated:

$$\text{MAD} = \text{median}\left(|x_i - \tilde{x}|\right) \tag{4.7}$$

Where $\tilde{x}$ is the median of the peer group. The Modified Z-score (z_i) for each company i is defined as:

$$z_i = 0.6745 \times \frac{x_i - \tilde{x}}{\text{MAD}} \tag{4.8}$$

The constant 0.6745 serves as a scale factor, ensuring that the expected value of the robust Z-score matches standard normal Z-scores for normally distributed datasets.

To evaluate the standardized deviation of each corporate observation, the outlier screening is formulated as a hypothesis test:

$$\mathbf{H}_0 : \textbf{The deviation from the median is within normal variation} \tag{4.9}$$

$$\mathbf{H}_1 : \textbf{The deviation from the median is statistically extreme} \tag{4.10}$$

We fail to reject H_0 for normal observations (inliers). If the absolute modified Z-score $|z_i|$

exceeds the critical threshold θ_{zscore}, the null hypothesis is rejected, and the ratio is flagged as a univariate outlier:

$$|z_i| > \theta_{\text{zscore}} \tag{4.11}$$

Where θ_{zscore} is the threshold parameter (default 3.50).

The mathematical symbols and operational parameters for robust Modified Z-score calculation are defined in Table 4.2.

Table 4.2: Robust Modified Z-Score Variables

Variable / Symbol	Operational Definition and Description
$\tilde{\boldsymbol{x}}$	Median of the corporate peer group for the evaluated financial ratio.
MAD	Median Absolute Deviation, a robust outlier-resistant dispersion measure.
$\mathbf{z}_i$	Robust modified Z-score calculated for company i using median and MAD.
$\boldsymbol{\theta}_{\textbf{zscore}}$	Severity threshold (default 3.50) for robust Z-score outlier flagging.

4.3 Isolation Forest (Multivariate Anomaly Detection)

Univariate outlier detection methods evaluate each ratio independently. However, financial reporting irregularities often manifest as multi-dimensional structural anomalies across multiple ratios simultaneously.

The Isolation Forest algorithm isolates anomalies by randomly partitioning high-dimensional feature spaces. Because anomalous points require fewer partitions to isolate, they appear closer to the root of the decision trees.

The anomaly score (s) for a sample x is defined as:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \tag{4.12}$$

Where $h(x)$ is the path length to isolate sample x, $E(h(x))$ is the average path length across all trees, and $c(n)$ is the average path length of an unsuccessful search in a Binary Search Tree:

$$c(n) = 2\ln(n-1) + 0.5772156649 - \frac{2(n-1)}{n} \tag{4.13}$$

An anomaly score close to 1.0 indicates a highly anomalous structural profile across all

financial dimensions.

To classify corporate profiles based on joint financial ratios, we formulate multivariate anomaly detection as a decision-theoretic hypothesis test:

$$\mathbf{H}_0 : \textbf{The sample exhibits a typical joint distribution across features} \tag{4.14}$$

$$\mathbf{H}_1 : \textbf{The sample exhibits a structurally anomalous joint distribution} \tag{4.15}$$

Where τ is the critical decision threshold (set to 0.5 by default, which signifies that the expected path length matches the average path length of an unsuccessful search in a binary search tree). Under H_0, the sample is classified as a normal corporate profile. If the anomaly score is greater than or equal to τ, the null hypothesis is rejected, and the corporate entity is classified as a multivariate anomaly.

The mathematical parameters governing the tree-isolation metrics are summarized in Table 4.3.

Table 4.3: Isolation Forest Anomaly Detection Variables

Variable / Symbol	Operational Definition and Description
$\mathbf{s}(\mathbf{x}, \mathbf{n})$	Anomaly score for sample x across the decision tree ensemble.
$\mathbf{n}$	Total number of corporate samples / peer group size for tree partitioning.
$\mathbf{h}(\mathbf{x})$	Traversed path length (number of edges) to isolate sample x in a tree.
$\mathbf{E}(\mathbf{h}(\mathbf{x}))$	Expected path length $h(x)$ across the entire Isolation Forest ensemble.
$\mathbf{c}(\mathbf{n})$	Average path length of an unsuccessful search in a BST of size n.

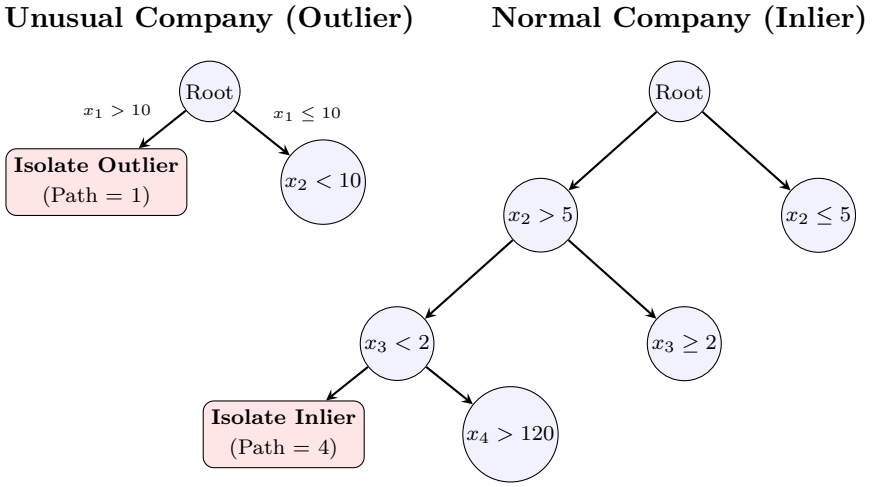


Figure 4.1: Multivariate Anomaly Isolation via Partition Tree Path Lengths

4.4 K-Means Clustering for Peer Archetypes

Unsupervised K-Means clustering is deployed to segment the peer set into distinct corporate archetypes based on a three-dimensional feature vector $\mathbf{X}$:

$$\mathbf{X} = [\text{Operating Margin}, \text{Debt-to-Equity}, \text{Asset Turnover}] \tag{4.16}$$

The algorithm groups the scaled feature space into $K = 3$ clusters by minimizing the within-cluster sum of squares (WCSS):

$$\arg\min_{\mathbf{S}} \sum_{j=1}^{3} \sum_{\mathbf{x}_i \in S_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2 \tag{4.17}$$

Centroids ($\boldsymbol{\mu}_j$) are mapped back to their original dimensions to determine corporate profiles:

- High Margins + Low Debt + Stable Turnover → **Elite Market Leader**
- Low Margins + High Debt → **Debt-Strained Underperformer**
- High Turnover + Low Margins → **High-Volume Bootstrapper**
- High Margins + Slower Turnover → **Asset-Heavy Profit Leader**
- Average Margins + Standard Debt + Standard Turnover → **Balanced Sector Participant**

The multidimensional parameters and vector spaces utilized in K-Means clustering are defined in Table 4.4.

Table 4.4: K-Means Clustering Archetype Variables

Variable / Symbol	Operational Definition and Description
$\mathbf{X}$	Three-dimensional feature vector of key financial operational dimensions.
$\mathbf{K}$	Total number of target corporate cluster archetypes (default set to 3).
$\mathbf{S}$	Partition set containing the corporate entities assigned to each cluster.
$\boldsymbol{\mu}_j$	Multidimensional centroid vector representing the profile of cluster j.

4.5 TallySync Double-Entry Bridge Ingestion

The *Tally Bridge* converts raw ERP ledger dumps and transaction journals into standard financial ratios. To prevent memory issues when processing large-scale journal logs, the engine streams files in chunks of 50,000 records. Because accounting entries can have varying sign behaviors depending on their category, the system checks ledger classifications: for credit-nature accounts, the balance is calculated as 'credit - debit', while for debit-nature accounts, it is computed as 'debit - credit'.

4.6 Algorithms Pseudocode

To facilitate the practical deployment and computational replication of our analytical framework, this section formalizes the core processes as structured, language-agnostic pseudocode. Specifically, we present the algorithmic logic governing our univariate outlier detection and baseline masking engine, which maintains statistical integrity across noisy peer distributions, followed by the TallySync double-entry journal processing pipeline designed to securely aggregate raw transaction logs. To facilitate cross-referencing and trace the exact operations, the primary variables and notations deployed across the pseudocodes are defined in Table 4.5.

Table 4.5: Algorithms Pseudocode Mathematical Symbols

Variable / Symbol	Operational Definition and Description
$\mathbf{R}, \mathbf{C}$	Input ratio matrix ($N \times M$) and output cleaned/masked ratio matrix.
$\mathbf{O}$	Outlier database tracking tagged anomalies in $\{(company, ratio, value)\}$.
$\mathbf{V}, \mathbf{Z}$	Non-null value vector and computed modified Z-scores for a ratio column.
$\mathbf{LB}, \mathbf{UB}$	Calculated Tukey outlier boundaries (Lower Bound and Upper Bound).
dr, cr	Debit and credit ledger transaction amounts from streamed journal lines.
$\mathbf{P}$	Hash-map storing aggregated double-entry balances for financial pillars.

4.6.1 Univariate Outlier Detection and Baseline Masking

The outlier detection procedure (detailed in Algorithm 1) robustly identifies and isolates anomalies in financial ratio distributions. The algorithm operates as follows:

1. **Data Cleaning**: For each financial ratio column, it drops any missing values to extract a clean vector V.

2. **Statistical Thresholding**: It calculates the Tukey IQR fences (Lower Bound LB, Upper Bound UB) and robust Modified Z-scores based on the Median Absolute Deviation

(MAD) of the distribution.

3. **Univariate Anomaly Flagging**: For each company, it checks if its ratio value v falls outside the Tukey fences (iqr_flag) or if its Modified Z-score exceeds the threshold (z_flag).

4. **Outlier Masking**: If an anomaly is detected, the company is added to the outlier database O, and the clean copy matrix C masks this specific cell as NaN (Not a Number). This prevents contaminated entries from skewing the peer group median and dispersion baselines.

Algorithm 1 Univariate Outlier Detection and Baselines

procedure DETECTOUTLIERS(R, k, θ)
  $O \leftarrow \emptyset$; $C \leftarrow R$ ▷ *Initialize outlier set and clean copy*
  for each ratio column j in R **do**
    $V \leftarrow \text{dropna}(R[:, j])$
    if $\text{length}(V) < 3$ **then continue**
    $Q_1, Q_3 \leftarrow \text{percentile}(V, 25), \text{percentile}(V, 75)$
    $\text{IQR} \leftarrow Q_3 - Q_1$
    $\text{LB}, \text{UB} \leftarrow Q_1 - k \cdot \text{IQR}, Q_3 + k \cdot \text{IQR}$
    $\tilde{v} \leftarrow \text{median}(V)$; $\text{MAD} \leftarrow \text{median}(|V - \tilde{v}|)$
    if $\text{MAD} = 0$ **then**
      $\sigma \leftarrow \text{std}(V)$; $Z \leftarrow (V - \text{mean}(V))/\sigma$
    else
      $Z \leftarrow 0.6745 \times (V - \tilde{v})/\text{MAD}$
    end if
    for each company index i in R **do**
      $v \leftarrow R[i, j]$
      $\text{iqr_flag} \leftarrow (v < \text{LB}) \vee (v > \text{UB})$; $\text{z_flag} \leftarrow |Z[i]| > \theta$
      if iqr_flag $\vee$ z_flag **then**
        $O \leftarrow O \cup \{(i, j, v)\}$; $C[i, j] \leftarrow \text{NaN}$ ▷ *Mask outlier cell*
      end if
    end for
  end for
  return O, C
end procedure

4.6.2 TallySync ERP Double-Entry Journal Aggregator

The TallySync ingestion bridge (detailed in Algorithm 2) acts as a high-throughput transaction engine, converting raw double-entry ledger streams into structured, standardized financial statements. The pipeline operates as follows:

1. **Ledger Mapping**: It maps raw ledger accounts from the ERP database to standard

public statement pillars (e.g. *revenue, debt, assets, payables*) using standard category classifications.

2. **Streamed Ingestion**: To prevent memory strain, transaction journals are processed in chunks of 50,000 logs.

3. **Sign-Nature Correction**: Transactions are aggregated according to the double-entry accounting nature of the pillar. For credit-nature categories, balances accumulate as ‘credit - debit‘, while debit-nature categories accumulate as ‘debit - credit‘.

4. **Ratio Ingestion**: Once all transactional journals are aggregated, the Trial Balance is resolved and derived into the standard 30+ canonical financial ratios.

Algorithm 2 TallySync ERP Double-Entry Journal Aggregator

```
procedure ProcessTally(J_path, L_path)
    G ← load_group_mappings(); M ← ∅; Master ← read_csv(L_path)        ▷ Init maps
    for each row in Master do
        ledger, group ← row['NAME'], row['PRIMARYGROUP']
        M[ledger] ← G.get(group, "EXCLUDE")
    end for
    P ← initialize_pillars_to_zero(); CR_Nature ← {"revenue", "payables", "liabilities", "debt", "equity"}
    Stream ← stream_csv(J_path, chunk_size = 50000)
    for each chunk in Stream do
        for each row in chunk do
            ledger ← row['PARTICULARS']; pillar ← M.get(ledger, "EXCLUDE")
            if pillar = "EXCLUDE" then continue
            dr, cr ← float(row['DEBIT'], 0.0), float(row['CREDIT'], 0.0)
            if pillar ∈ CR_Nature then
                P[pillar] ← P[pillar] + (cr − dr)
            else
                P[pillar] ← P[pillar] + (dr − cr)
            end if
        end for
    end for
    R ← calculate_ratios_from_pillars(P)
    return R
end procedure
```

Chapter 5

Implementation

5.1 Technologies Used

The *FinMetrics Suite* is developed entirely in Python 3.10+, utilizing standard scientific computing, data science, and machine learning libraries.

5.2 Tools and Libraries

The package dependencies and their structural roles are defined below:

- **Pandas (v2.1.1)**: Used for high-speed tabular operations, statement transpositions, mapping Tally ERP ledgers, and streaming journal logs in chunks.
- **NumPy (v1.26.0)**: Provides vector operations, array manipulation, and robust calculations to avoid zero-division errors.
- **SciPy (v1.11.3)**: Supplies scientific functions, specifically ‘scipy.stats.percentileofscore‘ for ranking companies.
- **Scikit-Learn (v1.3.1)**: Powers the machine learning engines: ‘RobustScaler‘ for scaling, ‘IsolationForest‘ for anomaly detection, and ‘KMeans‘ for peer clustering.
- **Streamlit (v1.27.2)**: Provides the web frontend framework, handling interactive dashboards, stateful re-runs, and file upload structures.
- **Plotly (v5.17.0)**: Generates interactive visualizations (scatter plots, rank bar charts, heatmaps).
- **yfinance (v0.2.31)**: Handles connections to the Yahoo Finance API to fetch statements and market metrics.
- **openpyxl & XlsxWriter (v3.1.2)**: Generates styled multi-sheet Excel reports.

- **google-generativeai (v0.3.1)**: Manages Gemini model integration for natural-language stock discovery and diagnostic chats.

5.3 Development Workflow and Methodology

The system was designed and developed following an iterative, modular development lifecycle. Decoupling the data harvesters, standardizing parsers, and statistical scoring kernels allowed for thorough testing and validation before building the frontend app layer. The sequential phases of our development workflow are illustrated in Figure 5.1.

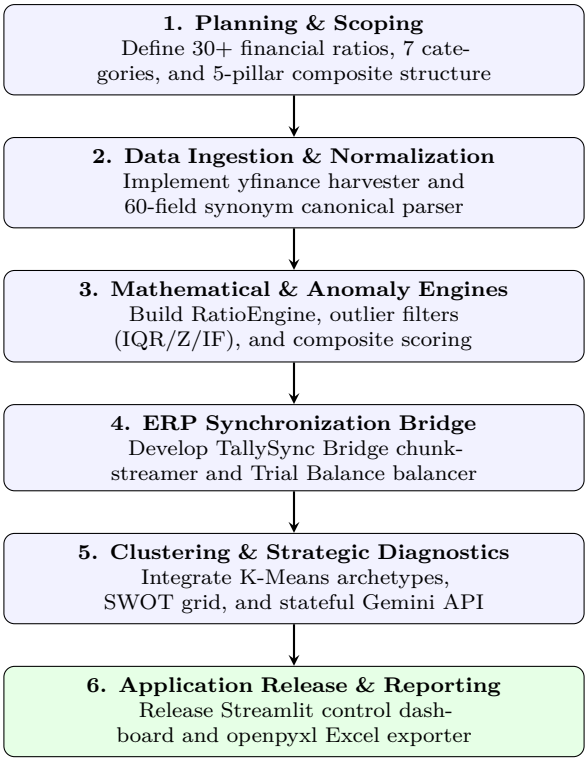


Figure 5.1: Iterative Modular Development Workflow

5.4 Code Structure Explanation

The project directory is structured as a decoupled Python package:

```
finoutlier/
|-- app.py                    # Streamlit dashboard UI
|-- requirements.txt          # Package dependencies
|
```

```
|-- modules/
|   |-- __init__.py
|   |-- sector_loader.py        # Pre-mapped sectors and tickers
|   |-- data_fetcher.py         # yfinance API scraper and cache
|   |-- financial_parser.py     # Standardizes statements via aliases
|   |-- ratio_engine.py         # Computes 30+ ratios and YoY growth
|   |-- outlier_engine.py       # Tukey, Robust Z, Isolation Forest
|   |-- benchmark_core.py       # Core baseline and pillar scoring
|   |-- tally_bridge.py         # ERP double-entry aggregator
|   |-- client_comparator.py    # Maps client metrics to baselines
|   |-- benchmark_classifier.py# Assigns health and SWOT labels
|   |-- report_generator.py     # Styled Excel workbook builder
|   |-- ai_handler.py           # Gemini AI API interface
|   |-- utils.py                # Math and cache utility functions
|
|-- data/
|   |-- raw/                    # Cached raw statements (Parquet)
|   |-- processed/              # Cleaned standardized statements
|   |-- cache/                  # Stored sector baseline JSONs
|
|-- outputs/
    |-- reports/                # Styled Excel reports
    |-- tables/                 # CSV tables
```

Listing 5.1: FinMetrics Suite Folder Structure

5.5 Module-wise Implementation Details

The key modules are implemented as isolated classes:

5.5.1 Financial Standardizer (financial_parser.py)

The standardizer uses a nested dictionary structure containing alias mappings for each statement. It scans the rows, maps variants using the alias list, removes duplicate metrics, and converts all entries to numeric types:

```
def _resolve_alias(self, raw_name: str, alias_dict: dict) -> str:
    norm = raw_name.strip().lower()
    for canonical, variants in alias_dict.items():
        for v in variants:
            if v.lower() == norm:
```

```
                return canonical
    return None
```

Listing 5.2: Alias Resolver in FinancialParser

5.5.2 Outlier Scoring (outlier_engine.py)

Calculates severity scores and directional flags. It computes IQR fences and robust standard deviations to flag statistical anomalies:

```
def _severity(self, val: float, lower: float, upper: float, iqr: float, z: float) -> float:
    iqr_score = 0.0
    if iqr > 0:
        if val < lower:
            iqr_score = (lower - val) / iqr
        elif val > upper:
            iqr_score = (val - upper) / iqr
    z_score = max(0.0, abs(z) - self.zscore_threshold)
    return round(0.6 * iqr_score + 0.4 * z_score, 3)
```

Listing 5.3: Severity Score Formulation

5.5.3 Tally Bridge Streamer (tally_bridge.py)

Processes large-scale journal CSV files in chunks using Pandas:

```
for chunk in pd.read_csv(self.jv_path, encoding="latin1", chunksize=self.chunk_size):
    for _, row in chunk.iterrows():
        ledger = str(row.get("PARTICULARS", "")).strip()
        pillar = ledger_map.get(ledger, "EXCLUDE")
        if pillar == "EXCLUDE":
            continue
        debit  = float(row["DEBIT"]) if not pd.isna(row.get("DEBIT")) else 0.0
        credit = float(row["CREDIT"]) if not pd.isna(row.get("CREDIT")) else 0.0

        if pillar in _CREDIT_NATURE:
            balance = credit - debit
        else:
```

```
            balance = debit - credit
        pillars[pillar] += balance
```

Listing 5.4: Journal Streaming in TallyBridge

5.6 System Component Integration

To illustrate the orchestration pipeline, synchronous execution flow, and architectural relationships between the frontend UI, the Tally ERP translation bridge, the percentile comparator, and the outlier scoring engine, a detailed system architecture and orchestration flow diagram of the Phase 2 workflow is provided in Figure 5.2.

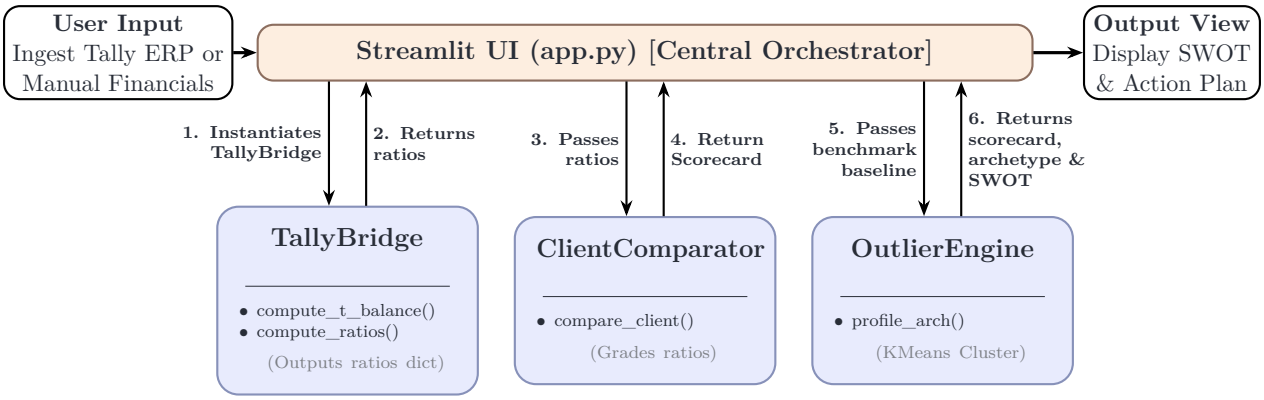


Figure 5.2: System Architecture and Orchestration Pipeline of Phase 2 Workflow

Chapter 6

Results and Discussion

6.1 Experimental Setup

The analytical engine was tested on a Windows 11 platform equipped with an Intel Core i7 processor and 16 GB of RAM. The evaluation was split into two testing protocols:

1. **Public Baseline Performance**: Benchmarked 13 public market sectors, with peer group sizes ranging from 5 to 29 companies.
2. **Private Client Integration**: Processed a Tally ERP dataset containing 1,385 ledger accounts and a multi-thousand row Journal Voucher log.

6.2 Results Analysis

The TallySync bridge successfully processed the raw double-entry transaction log and generated a balanced Trial Balance:

- **Total Unique Ledgers Ingested**: 1,385
- **Total Aggregated Debit Balances**: 590,856,997.09 INR
- **Total Aggregated Credit Balances**: 590,856,997.09 INR
- **Trial Balance Difference**: 0.00000213 INR (rounding precision limit)
- **Pillar Mapping**: Mapped ledger accounts to their corresponding accounting categories (e.g. standardizing inventory, receivables, debt).

6.3 Outlier Masking Evaluation

To evaluate the impact of outlier masking, we compared the descriptive baseline statistics calculated before and after running the statistical filters (Table 6.1).

Table 6.1: Impact of Outlier Masking on Peer Group Statistics

Metric	Before Outlier Masking	After Outlier Masking
Company 1 (Operating Margin %)	18.2%	18.2%
Company 2 (Operating Margin %)	19.5%	19.5%
Company 3 (Operating Margin %)	17.1%	17.1%
Company 4 (Operating Margin %)	85.0% (Anomalous)	masked (NaN)
Company 5 (Operating Margin %)	16.5%	16.5%
Company 6 (Operating Margin %)	-45.0% (Anomalous)	masked (NaN)
Company 7 (Operating Margin %)	19.0%	19.0%
Sector Median	18.2%	18.2% (Robust)
Sector Mean	18.61%	18.06% (Cleaned)
Sector Standard Deviation	39.54%	1.28% (Cleaned)

Without masking, the anomalous margins of Company 4 (85.0%) and Company 6 (−45.0%) skewed the standard deviation to 39.54%, which diluted the comparative boundaries. Masking the outlier cells restored the standard deviation to 1.28%, protecting the benchmark baseline.

6.4 Interpretation of Client Results

The system benchmarked a private FMCG client against the public peer baseline. The client's financial profile revealed a clear strategic contradiction (Figure 6.1):

- **Profitability**: Operating Margin in the 85th percentile (assigned an *"Elite"* health label).
- **Liquidity**: Current Ratio in the bottom 10th percentile (assigned a *"Critical"* health label).
- **Interpretation**: The client possesses strong pricing power and cost controls, but faces a severe liquidity squeeze. A deep-dive into working capital velocity revealed slow inventory turnover, indicating cash is locked up in unsold stock.

6.4.1 Strategic SWOT Recommendations and Action Plan

Based on the synthesized 2x2 SWOT grid and the corporate archetype classification, a targeted operational action plan was developed for the FMCG client:

- **Optimize Working Capital Velocity (Liquidity Strategy)**: Transition to a Just-

In-Time (JIT) replenishment model for slow-moving inventory and run promotional clearing cycles to convert stagnant stock into operational cash.

- **Leverage Pricing Power (Profitability Strategy)**: Utilize the client's high Operating Margin (85th percentile) to negotiate shorter credit terms or advance payments with distributors, accelerating accounts receivable collections.
- **Extend Accounts Payable Cycles**: Extend vendor payment terms (Days Payable Outstanding) from 30 to 45 days to preserve operational cash flow without incurring additional short-term debt.
- **Secure Revolving Credit Lines**: Establish a pre-approved revolving credit line to cover immediate obligations (e.g., payroll) while inventory velocity and collection reforms take effect.

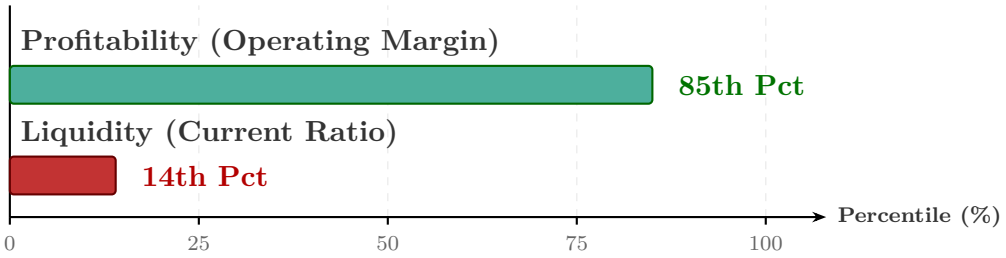


Figure 6.1: Client Financial Health Scorecard (Percentile Rankings)

6.5 Performance Analysis

The execution times of the core pipeline stages are summarized in Table 6.2.

Table 6.2: Execution Times for Pipeline Stages

Pipeline Stage	Execution Time (Seconds)
Data Harvesting (15 Peers, uncached)	14.50 s
Data Ingestion (15 Peers, cached)	1.20 s
Tally ERP Aggregation (100k rows)	4.20 s
Outlier Detection & Core Scoring	0.45 s
UML/K-Means Clustering	0.25 s
Excel Report Generation	1.10 s

The caching layer reduces statement ingestion times by over 90%, making the application highly responsive.

Chapter 7

Conclusion and Future Enhancements

7.1 Final Outcomes

The *FinMetrics Suite* successfully addresses key challenges in quantitative financial analysis, outlier detection, and public-private benchmarking:

- Its standardizer successfully resolves statement naming variations across companies, building consistent peer datasets automatically.
- By screening and masking statistical outliers, the system protects peer group medians and standard deviations, establishing objective comparative baselines.
- The TallySync bridge provides a seamless ingestion layer for private enterprise managers, stream-processing double-entry journal vouchers to balance Trial Balances and derive standard ratios.
- Integrating unsupervised archetype clustering, rule-based SWOT matrices, and Gemini-driven interactive diagnostics allows business owners to easily translate raw data into clear, actionable business strategies.

7.2 Contributions

The primary contributions of this project include:

1. **Double-Entry ERP Translation Layer**: Developed an ERP translation utility capable of processing raw double-entry transaction ledgers, checking credit-nature signs, and balancing a Trial Balance.
2. **Outlier-Resistant Benchmarks**: Designed a baseline mapping process that masks outlier cells before calculating descriptive peer statistics, protecting peer medians and quartiles from extreme data points.

3. **Unsupervised Archetype Segmentations**: Integrated K-Means clustering to group companies into distinct corporate archetypes, providing private enterprise managers with clear, objective strategic context.

7.3 Future Scope

While the platform provides robust analytical performance, several enhancements are planned for future versions:

- **Multi-Year Trend Analysis**: Extend the ERP ingestion engine to process multi-year journal logs, enabling private enterprises to track long-term financial trends.
- **Direct Database Connectors**: Integrate API connections for major ERP systems (e.g. SAP, Oracle, Zoho Books), removing the need to export and upload manual CSV files.
- **Real-Time Sector Alerting**: Implement automated alert systems that monitor selected sectors and notify analysts when a company's ratios trigger extreme outlier or solvency warnings.
- **Predictive Analytics**: Integrate predictive models (such as ARIMA or Prophet) to forecast ratio trends, providing forward-looking strategic insights.

Bibliography

[1] Aswath Damodaran. *Investment valuation: Tools and techniques for determining the value of any asset*, volume 666. John Wiley & Sons, 2012.

[2] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. pages 413–422, 2008.

[3] James MacQueen. Some methods for classification and analysis of multivariate observations. 1(14):281–297, 1967.

[4] John W Tukey. Exploratory data analysis. *Reading, Mass.*, 2:231, 1977.

Appendix A

Core Source Code

A.1 Outlier Filtering Logic

The python source code for univariate outlier screening inside *OutlierEngine* is shown below:

```
def detect(self, ratio_matrix: pd.DataFrame) -> pd.DataFrame:
    records = []
    ratio_cols = [c for c in ratio_matrix.columns if c not in EXCLUDE_COLS
   ]

    for ratio in ratio_cols:
        col = ratio_matrix[ratio].dropna()
        if len(col) < self.min_peers:
            continue

        q1, q3 = col.quantile(0.25), col.quantile(0.75)
        iqr = q3 - q1
        lower = q1 - self.iqr_multiplier * iqr
        upper = q3 + self.iqr_multiplier * iqr
        median = col.median()
        mean = col.mean()

        # Robust Z-Scores
        mad = (col - median).abs().median()
        if mad == 0:
            std = col.std()
            zscore_series = (col - mean) / std if std != 0 else pd.Series
   (0.0, index=col.index)
        else:
            zscore_series = 0.6745 * (col - median) / mad

        for ticker in ratio_matrix.index:
            val = ratio_matrix.at[ticker, ratio]
            if pd.isna(val):
                continue

```

```
            iqr_flag = bool(val < lower or val > upper)
            z = float(zscore_series.get(ticker, 0.0))
            zscore_flag = bool(abs(z) > self.zscore_threshold)

            if iqr_flag or zscore_flag:
                dev_pct = ((val - median) / abs(median) * 100) if median
!= 0 else np.nan
                severity = self._severity(val, lower, upper, iqr, z)
                direction = self._direction(ratio, val, median)
                is_positive = self._is_positive_outlier(ratio, direction)

                records.append({
                    "ticker": ticker,
                    "ratio": ratio,
                    "value": round(val, 4),
                    "sector_median": round(median, 4),
                    "zscore": round(z, 3),
                    "iqr_flag": iqr_flag,
                    "zscore_flag": zscore_flag,
                    "deviation_pct": round(dev_pct, 2) if not pd.isna(
dev_pct) else np.nan,
                    "severity_score": round(severity, 2),
                    "is_positive_outlier": is_positive,
                })

    return pd.DataFrame(records)
```

Listing A.1: Univariate Outlier Detection Code

Appendix B

Tally ERP Ingestion Formats

The required CSV structures for uploading private enterprise ledger groups and double-entry transaction journals in the *TallySync* bridge are defined below:

B.1 Ledger Master CSV Format

The Ledger Master file maps local transaction accounts to their primary Tally accounting groups:

```
1 CLIENT_ID,COMPANY_NAME,FINANCIAL_YEAR,NAME,PARENT,PRIMARYGROUP,OPENING,
    CLOSING,GUID,SYNC_TIME
2 10000_VSM,VIBRANT SUPERMARKET,2025-26,0% CST SALES,CST T/O,Sales Accounts
    ,0.0,0.0,783aa38d-...,15-04-26 20:26
3 10000_VSM,VIBRANT SUPERMARKET,2025-26,0% Integrated Supply,Purchase
    Accounts,Purchase Accounts,0.0,-6000907.42,783aa38d-...,15-04-26 20:26
```

Listing B.1: Ledger Master CSV Format

B.2 Journal Voucher CSV Format

The Journal Voucher file streams the actual credit and debit transaction journal voucher records:

```
1 CLIENT_ID,COMPANY_NAME,FINANCIAL_YEAR,DATE,PARTICULARS,VCHTYPE,VCHNO,DEBIT
    ,CREDIT,VOUCHERKEY,NARRATION
2 10000_VSM,VIBRANT SUPERMARKET,2025-26,1-Apr-25,KISHOREKUMAR,GST Common
    Invoices,KIS-01/25-26,0.0,2000.0,783aa38d-...,Being Rent
3 10000_VSM,VIBRANT SUPERMARKET,2025-26,1-Apr-25,Rent,GST Common Invoices,
    KIS-01/25-26,-2000.0,0.0,783aa38d-...,Being Rent
```

Listing B.2: Journal Voucher CSV Format